



Programación de Servicios y Procesos

TEMA 4. GENERACIÓN DE SERVICIOS DE RED

4.1. Programación y verificación de aplicaciones cliente de protocolos estándar

Las aplicaciones cliente son aquellas que realizan peticiones a un servidor, solicitando cualquier tipo de recurso o información que esté alojado en el servidor.

El ejemplo más claro de aplicaciones cliente son los **navegadores**, a través de los cuales se realizan la mayor parte de las peticiones. Las aplicaciones instaladas en cualquier tipo de dispositivo que requieran de Internet para su funcionamiento son, de igual manera, un tipo de cliente.

Mediante Java se pueden crear aplicaciones que se conecten a distintos servicios, como pueden ser clientes FTP, Telnet o SMTP, entre otras.

El modelo TCP/IP está compuesto por cuatro capas, donde la capa de aplicación maneja protocolos de alto nivel que implementa servicios como:

- Conexión remota → Telnet
- Correo electrónico → SMTP
- Acceso a ficheros remotos → FTP, NFS, TFTP
- Resolución de nombres de ordenadores → DNS, WINS
- World Wide Web → HTTP

Capas TCP/IP	Capas y protocolos TCP/IP	
Aplicación	SMTP, Telnet, FTP, HTTP	NFS, SNMP, DNS
Transporte	TCP	UDP
Internet	IP	
Interfaz Red	Protocolos de subred	

A lo largo del capítulo se estudiarán las distintas clases que ofrece el API de Java para realizar este tipo de conexiones, y así poder lograr una comunicación con los distintos servicios.

1.1. Programación de servidores

Para dar servicio a las peticiones que realizan los clientes desde cualquier tipo de aplicación es necesario almacenar en una máquina todos los datos que se van a solicitar. Para esto se utilizan los servidores, que, como se ha comentado anteriormente, son equipos informáticos que dan servicio a otros equipos.

Debido al gran volumen de peticiones que pueden llegar a tener este tipo de equipos, los servidores se caracterizan por tener unas prestaciones técnicas muy superiores al resto de ordenadores que se comercializan.

Existen dos tipos de servidores:

- Servidor compartido: servidores con prestaciones muy altas, que son compartidos entre diferentes empresas por su elevado coste.
- Servidor dedicado: servidores únicos para cada empresa.

Los servidores son los encargados de proporcionar ciertos servicios a los clientes, es decir, de ofrecer una serie de programas que permiten gestionar los recursos:

- DNS: cuando un cliente solicita una dirección web desde un navegador, el servidor se encarga de determinar en qué lugar se encuentra esta web y le indica la respuesta al cliente.
- DHCP: cuando un cliente se conecta a un servidor es necesario conocer quién es el cliente a través de su IP. El protocolo DHCP configurado en el servidor permite asignar de manera dinámica una IP a cada cliente que se conecta. Al contrario que los clientes, los servidores siempre tienen una IP fija.
- FTP: protocolo de transferencia de archivos. Es un protocolo utilizado para poder enviar un fichero de un cliente a un servidor. El servidor debe ser configurado para poder permitir este tipo de transferencias. La mayor parte de los servidores permiten esto. FTP no lleva ningún tipo de encriptación, por lo que si se desea un servicio de transferencia encriptada es necesario utilizar SFTP.
- TELNET: protocolo que permite acceder a otro equipo remoto a través de la terminal. Actualmente lo más habitual es utilizar una de sus variantes que es SSH, que permite hacer uso de una conexión remota cifrada.

- HTTP: es uno de los protocolos más conocidos y más usados en las aplicaciones cliente. Permite hacer uso del intercambio de información en Internet (World Wide Web). Emplea un esquema de petición-respuesta, petición del cliente y respuesta del servidor web.
- NFS: este protocolo permite que distintos equipos que forman parte de una misma red puedan acceder a ficheros como si estuvieran almacenados de forma local en el equipo.
- SMTP: protocolo simple de transferencia de correo electrónico. Permite a los clientes el envío de correo entre distintos dispositivos, mientras que mediante los protocolos POP e IMAP se reciben.

4.2. Análisis de librerías de clases y componentes

- Telnet

- Constructores:

TelnetClient ()	Constructor por defecto. Su terminal type es VT100.
TelnetClient (String terminalType)	Constructor con el terminal type recibido por parámetro.

- Métodos:

void disconnect ()	Desconecta la sesión Telnet.
InputStream getInputStream ()	Devuelve el stream de entrada de la conexión Telnet.
OutputStream getOutputStream ()	Devuelve el stream de salida de la conexión Telnet.

Para más información:

<https://commons.apache.org/proper/commons-net/javadocs/api-3.6/org/apache/commons/net/telnet/TelnetClient.html>

- FTP

- Constructor:

FTPClient ()	Crea un cliente FTP
--------------	---------------------

- Métodos:

void disconnect ()	Cierra la conexión al servidor FTP.
void abort ()	Interrumpir una transferencia que se encuentra en proceso.
boolean deleteFile ()	Elimina un fichero del servidor FTP.
boolean changeToParentDirectory ()	Cambia al directorio padre de donde se encuentra.
boolean changeWorkingDirectory (String pathname)	Cambia el directorio de trabajo por el que se indica en la ruta.
boolean deleteFile (String pathname)	Elimina el archivo indicado.
FTPFile[] listDirectories ()	Devuelve la lista de directorios que se encuentran en el directorio de trabajo.
FTPFile[] listDirectories (String pathname)	Devuelve la lista de directorios que se encuentra en la ruta indicada.
FTPFile[] listFiles ()	Devuelve la lista de archivos que se encuentran en el directorio de trabajo.
FTPFile[] listFiles (String pathname)	Devuelve la lista de archivos que se encuentran en la ruta indicada.
String[] listNames ()	Devuelve la lista de nombres de los archivos que se encuentran en el directorio de trabajo.
String[] listNames (String pathname)	Devuelve la lista de nombres de los archivos que se encuentran en la ruta indicada.
boolean login (String username, String password)	Entra en el servidor FTP mediante el usuario y contraseña.
boolean logout ()	Cierra sesión en el servidor FTP.
boolean makeDirectory (String pathname)	Crea un nuevo directorio en la ruta especificada.
String printWorkingDirectory ()	Devuelve el nombre del directorio de trabajo.
boolean removeDirectory (String pathname)	Elimina el directorio especificado.

boolean rename (String from, String to)	Cambia el nombre de un fichero del servidor.
boolean retrieveFile (String remote, OutputStream local)	Recupera el contenido del fichero del servidor en un flujo de datos.
boolean storeFile (String remote, InputStream local)	Almacena el contenido del flujo en un fichero del servidor.

Para más información:

<https://commons.apache.org/proper/commons-net/apidocs/org/apache/commons/net/ftp/FTPClient.html>

Algunos de estos métodos devuelven un array de objetos de la clase FTPFile. Esta clase se utiliza para mostrar la información de los ficheros almacenados en un servidor FTP.

String getName ()	Devuelve el nombre de un fichero.
long getSize ()	Devuelve el tamaño del fichero en bytes.
int getType ()	Devuelve el tipo de fichero. Puede ser directorio, fichero o enlace simbólico.
String getUser ()	Devuelve el usuario propietario del fichero.
boolean isDirectory ()	Comprueba si es un directorio.
boolean isFile ()	Comprueba si es un fichero.
void setName (String name)	Establece el nombre de un archivo.
void setUser (String user)	Establece el usuario propietario de un archivo.
String toString ()	Devuelve en forma de cadena el contenido del fichero.

Para más información:

<https://commons.apache.org/proper/commons-net/apidocs/org/apache/commons/net/ftp/FTPFile.html>

- HTTP
- Constructor:

<code>URLConnection(URL u)</code>	Crea la conexión.
-----------------------------------	-------------------

- Métodos:

<code>abstract void disconnect ()</code>	Indica que serán improbables próximas peticiones al servidor.
<code>String getRequestMethod ()</code>	Devuelve el método de solicitud.
<code>int getResponseCode ()</code>	Devuelve el código del estado de un mensaje HTTP.
<code>String getResponseMessage ()</code>	Devuelve el mensaje de respuesta HTTP.

Para más información:

<https://docs.oracle.com/javase/7/docs/api/java/net/URLConnection.html>

- SMTP
- Constructor:

<code>SMTPClient ()</code>	Constructor por defecto.
<code>SMTPClient (String codificación)</code>	Se establece una codificación en el constructor.

- Métodos:

<code>boolean login ()</code>	Inicia sesión en el servidor SMTP.
<code>boolean login (String hostname)</code>	Inicia sesión en el servidor SMTP del host indicado.

<code>boolean logout ()</code>	Cierra la sesión con el servidor.
<code>Writer sendMessageData ()</code>	Envía el mensaje de correo.
<code>boolean verify (String username)</code>	Comprueba que la dirección de correo es válida.

Para más información:

<https://commons.apache.org/proper/commons-net/javadocs/api-3.6/org/apache/commons/net/smtp/SMTPClient.html>

4.3. Análisis de requisitos para servidores concurrentes

El avance de las tecnologías ha supuesto que la comunicación y la transferencia de datos a través de la red sea fundamental a nivel mundial. Esto supone que la infraestructura informática sea capaz de gestionar y procesar un gran número de peticiones a la vez.

Cuando se va a proceder a instalar y configurar un servidor, es esencial analizar cuál va a ser el servicio que va a ofrecer y cuántas conexiones será capaz de atender.

Si el servidor está destinado a recibir un número pequeño de procesos y cuya atención supone emplear una gran cantidad de recursos, es más aconsejable configurarlo como servidor iterativo. Este solo atenderá una petición y hasta que no finalice no cogerá la siguiente. Estos servidores suelen hacer uso del protocolo UDP.

Si el servidor tiene que soportar una gran cantidad de peticiones y además se tiene que garantizar que sean resueltas correctamente, es mejor hacer uso de un servidor concurrente. Este permite atender a varios clientes de forma simultánea. Estos servidores suelen hacer uso del protocolo TCP.

Ambos están continuamente esperando por la llegada de una petición por parte del cliente.

En la actualidad, la mayor parte de servidores hacen uso de los servidores concurrentes. Esto supone que no exista un colapso cuando se realizan miles de peticiones a un mismo servicio web, por ejemplo, desde diversos dispositivos.

4.4. Implementación de comunicaciones simultáneas

Existen diferentes tecnologías para llevar a cabo la comunicación entre cliente y servidor. A partir del mecanismo más básico que son los sockets existen algunas técnicas que permiten implementar dicha comunicación simultánea a un más alto nivel.

- RMI

La técnica más utilizada es la invocación a métodos remotos, RMI (Remote Method Invocation). Dicha invocación implica que el objeto solicitado estará en una red diferente. Consiste en la petición de un objeto por parte del cliente al servidor. El cliente debe realizar dicha petición cumpliendo con todos los parámetros definidos en el servidor para este servicio. Esta petición es procesada por el servidor que comprueba dicho acceso. El servidor, una vez verifica dicho acceso, envía la respuesta con el objeto solicitado al cliente.

Los parámetros obligatorios para realizar una invocación son:

- Objeto servidor o remoto: es el encargado de recibir la petición de acceso al método y la procesa.
- Objeto cliente: es el objeto que realiza la invocación al método remoto. Este realiza una petición al objeto servidor, quien le dará una respuesta.
- Método invocado: es el servicio invocado al que se accede y debe de ser realizado con todos sus parámetros. Este es quien comunica por mensajes la petición al objeto servidor.
- Valor de retorno: valor que se envía al objeto cliente como finalización del proceso de invocación.

- RPC

Existe una variante de esta técnica cuyo concepto es similar al RMI, es el RPC (Remote Procedure Call). La diferencia con el RMI es la estructura de programación de cada una de ellas. El RMI está basado en la programación orientada a objetos y el RPC en la programación estructurada. El cliente en el RPC realiza una petición directamente al método del servidor.

- Servicios web

Este tipo de técnicas de comunicación son estándares para cualquier tipo de acceso entre cliente y servidor. Además, existen algunas que definen la comunicación propia bajo entornos web, como son los servicios web. La mayor parte de los servicios web basan su comunicación mediante el uso de los protocolos SOAP y REST.

- SOAP (Simple Object Access Protocol): este protocolo define cómo dos objetos van a comunicarse. En SOAP esta comunicación se hará usando el lenguaje XML. Tanto los mensajes como el contenido de los mismos se realizan usando este lenguaje. Debido a que el lenguaje no cambia, y tanto cliente como servidor implementan esta misma comunicación, la petición se procesa de forma automática.
- REST (Representational State Transfer): estos servicios no se ven obligados a utilizar XML como lenguaje, sino que permiten otros como JSON, cuyo uso está muy expandido.

El uso de este viene definido por el tipo de operaciones que se quieren llevar a cabo.

Las diferentes operaciones son: POST, GET, PUT y DELETE.

Una de las ventajas que ofrece REST frente a SOAP es que los mensajes que son enviados ya definen todo lo necesario para ser procesados en el servidor, por lo que ni cliente ni servidor deben almacenar el estado de cada uno de ellos.

4.5. Verificación de la disponibilidad del servicio

A la hora de prestar un servicio es importante comprobar si este se encuentra disponible para el acceso de los clientes. En este caso, este procedimiento es tarea del administrador de red. Estos suelen estar sometidos a una disponibilidad de prácticamente un 100% durante mucho tiempo.

Esta exigencia puede provocar que ocurran fallos, por lo que este tipo de servidores cuentan con diferentes herramientas de monitorización de su estado. Así, es posible comprobar la disponibilidad de forma inmediata y realizar las acciones necesarias de mantenimiento.

Una de las formas que existen para comprobar su estado es realizar peticiones programadas a través de la URL del servicio web. Periódicamente se analizará el tiempo de respuesta del mismo y se comprobará el tiempo de carga de los recursos creados en dicho servicio.

Hoy en día existen servicios que contienen peticiones a datos que son especialmente sensibles a caídas. Existen, por tanto, herramientas que permiten garantizar una disponibilidad de prácticamente el 100% sin pérdidas de datos. Esto se conoce como alta disponibilidad.

Estos servidores permanecen activos junto con unas réplicas de sí mismos que permiten que, en caso de fallos, automáticamente se encargue el otro servidor de procesar los servicios.

Además, aquellos servicios sometidos a peticiones de forma masiva pueden hacer uso de balanceadores de carga, que se encargarán de distribuir las peticiones entre los distintos servidores para evitar un colapso de respuesta en el servicio. Esto, además de evitar colapsos, optimiza el rendimiento y añade una mayor velocidad y flexibilidad al servicio.

